Can you find Wally? Sem 1 Workshop 5

Teaching a CNN to draw a box around him



One of our generated training pictures. The green box is where we pasted Wally.

PART 1

A Quick Intro to Today's Activity

A quick tour, so you know where each step is going

The goal: one picture in, one box out



a 256 × 256 picture

our CNN
(WaldoFinder)

4 numbers

(x1, y1, x2, y2)

the top-left and bottom-right corners of a box around Wally

Green box = where Wally really is. Red box = where the model thinks he is.

The notebook in 7 steps

Step 1

Make our own pictures

paste Wally on 286 backgrounds, write down the box

Step 2

Feed them to PyTorch

Dataset + DataLoader serve batches of 32

Step 3

Build the CNN

4 conv blocks + a heatmap head

Step 4

Loss + optimiser

MSE says how wrong, Adam fixes it

Step 5

Train

10 epochs × 32 batches, about 2 min on GPU

Step 6

Score the boxes

IoU: how much the boxes overlap

Step 7

Test on new pictures

green vs red boxes, heatmaps, worst cases

Step 1: we make our own training pictures



286 backgrounds



5 Wallys, each tweaked every time (augmentation)

Paste with the alpha mask

Only Wally's own pixels are copied, not the white rectangle around him.

Tweak him every time

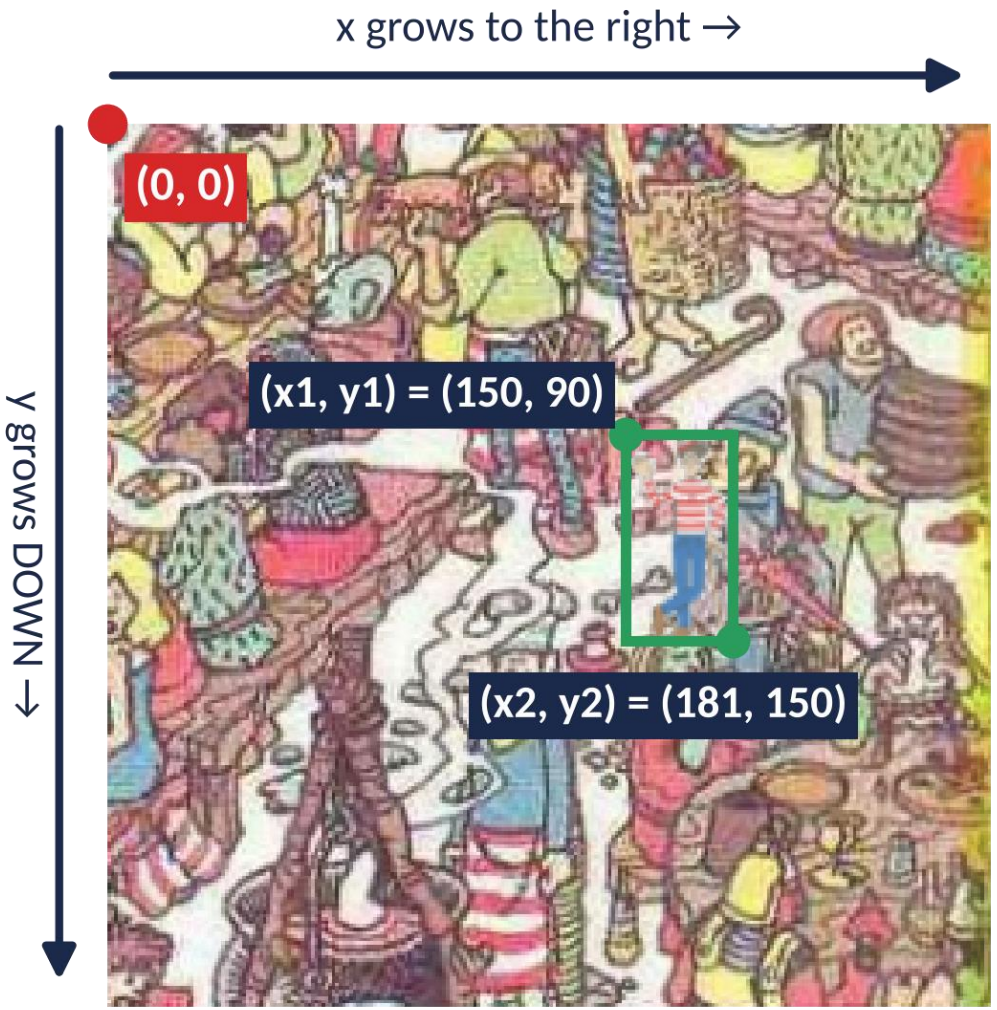
Small rotation, brightness and colour changes, so the model can't just memorise 5 pictures.

Labels for free

We placed him, so we already know the true box. No hand-labelling 5000 pictures!

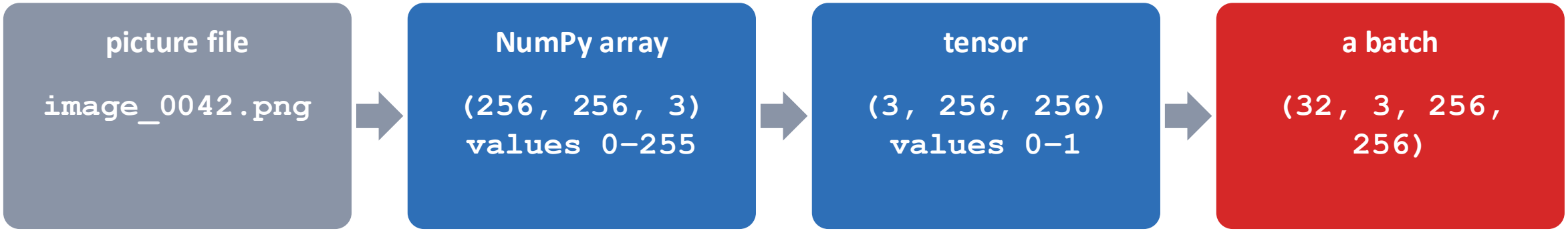
5000 pictures → 4000 for training, 1000 kept hidden for testing

Boxes and image coordinates



- **(0, 0) is the top-left corner,** not the bottom-left like in maths
- **x** counts pixels to the right, **y** counts pixels DOWN
- **A box is 4 numbers:** (x1, y1) top-left and (x2, y2) bottom-right
- **Width** = x2 − x1 = 31 px, **height** = y2 − y1 = 60 px
- **Inside the model** we divide by 256 so every number is between 0 and 1: (150, 90) becomes (0.59, 0.35)

Step 2: pictures become tensors, served in batches



(batch, channels, height, width). Most errors you'll hit today are shape errors, so print .shape when stuck!

32

images per batch

the weights are updated once per batch

32

batches per epoch

1000 images ÷ 32, rounded up (the last batch has 8)

320

updates in total

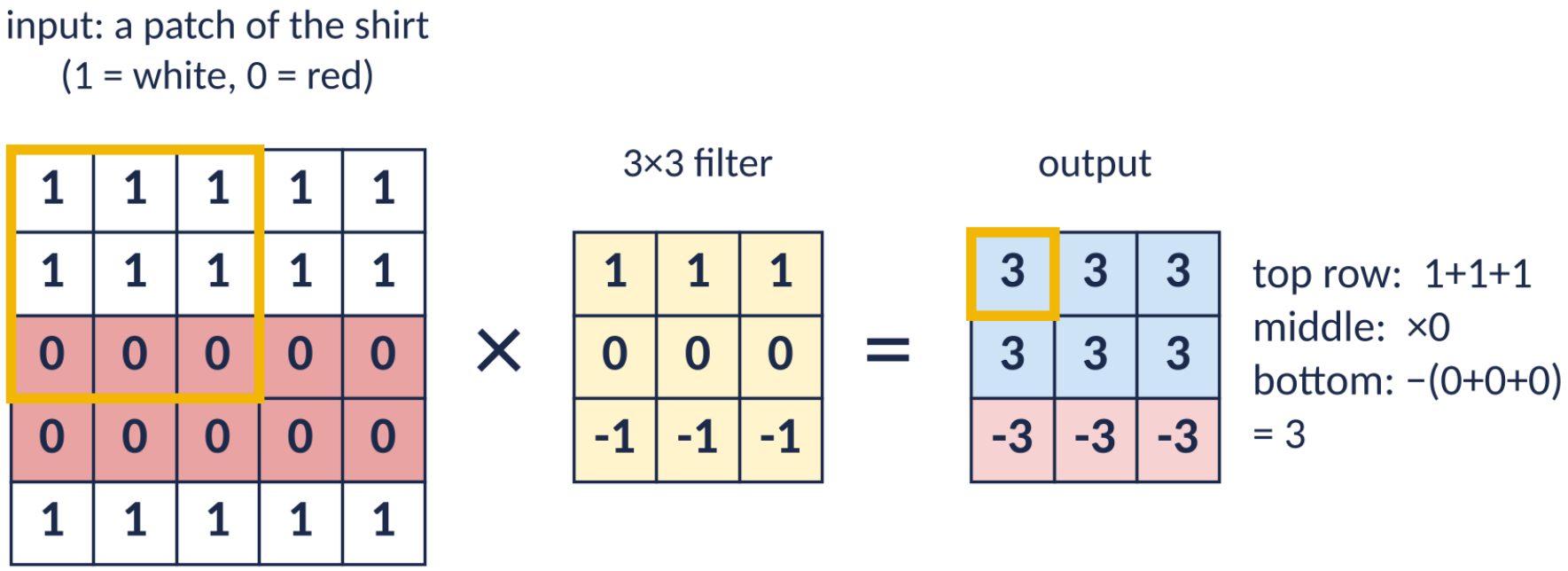
10 epochs × 32 batches

PART 2

How does a CNN see Wally?

Convolution, edges, padding, pooling, and one full layer

Convolution: slide, multiply, add



1 **Place**

Put the 3×3 filter on a 3×3 patch of the picture (gold square).

2 **Multiply and add**

Multiply each pixel by the filter number on top of it, then add all 9 up. That gives ONE number.

3 **Slide**

Move one pixel along and repeat. All the numbers together make a new grid: a feature map.

Horizontal and vertical edge detectors on Wally



input

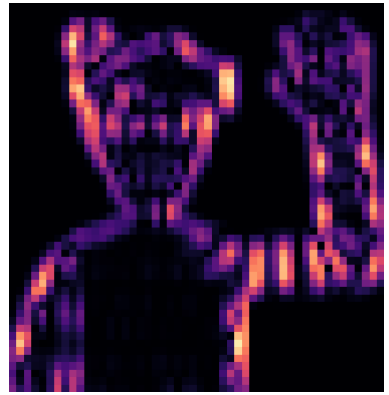
1	1	1
0	0	0
-1	-1	-1



horizontal-edge filter

the stripes on his shirt light up

1	0	-1
1	0	-1
1	0	-1



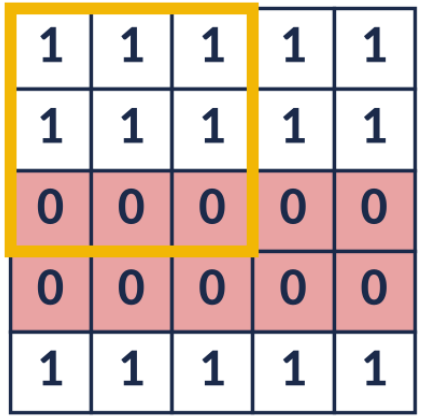
vertical-edge filter

the sides of his arm and face light up

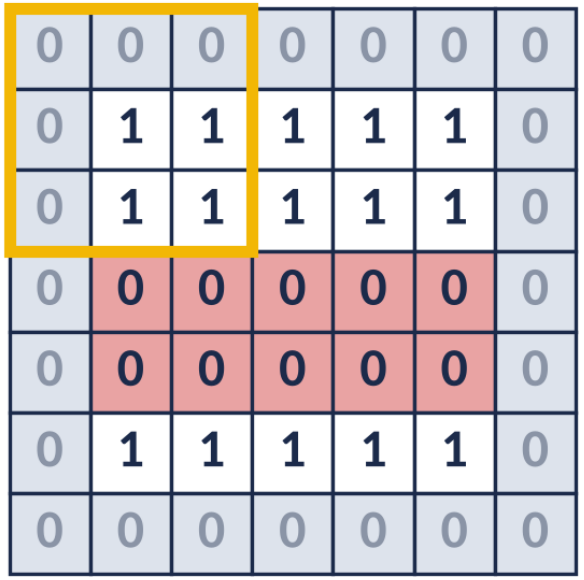
Bright = strong response. Same picture, different filter, different pattern found. In a CNN, nobody writes these filters by hand: the model learns them.

Padding: keeping Wally at the edge in view

no padding: 5×5 → 3×3
the filter can't be centred on edge pixels



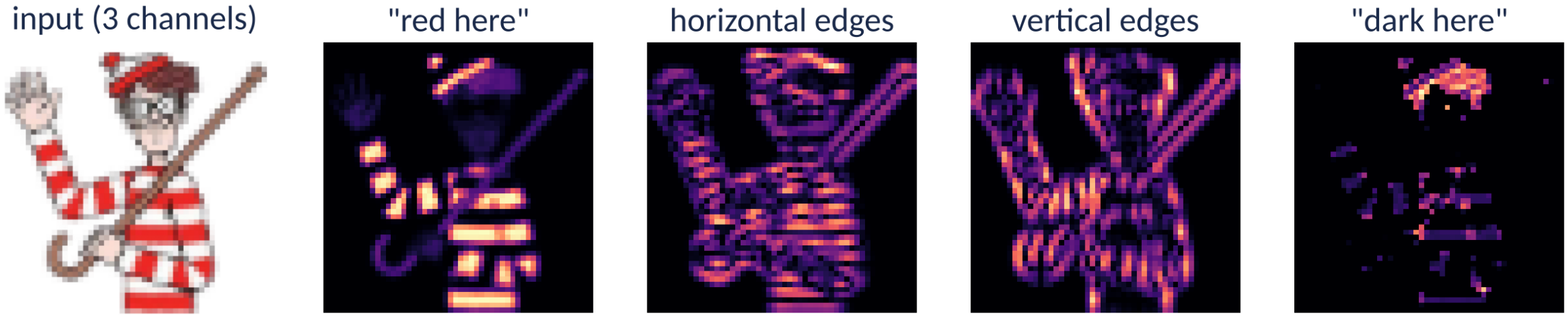
padding = 1: a border of 0s
5×5 → 5×5, every pixel gets a turn



- **The filter makes one number** for each position it can be centred on
- **Without padding** it can't sit on the edge pixels: 256 → 254, and Wally at the very edge gets ignored
- **padding = 1** adds a border of zeros, so 256 → 256 and every pixel gets a turn

```
nn.Conv2d(3, 16, kernel_size = 3, padding = 1)
```

Many filters, many channels



`nn.Conv2d(3, 16, kernel_size = 3)`

- **3 in:** red, green and blue
- **16 out:** 16 different filters, each making its own map (a channel)
- **each filter** is 3 × 3 × 3: it looks at all 3 colours at once

Deeper layers combine simple patterns

- **block 1:** edges, "red here"
- **block 2:** stripes, corners
- **blocks 3 and 4:** striped shirt, bobble hat, round glasses

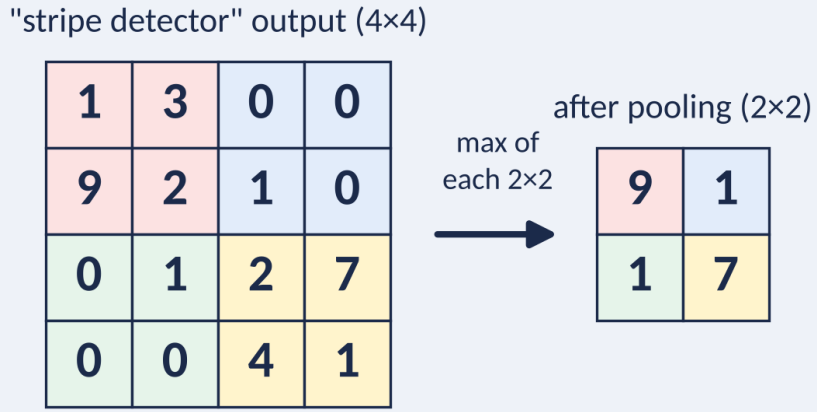
ReLU and max pooling

ReLU: keep the positives



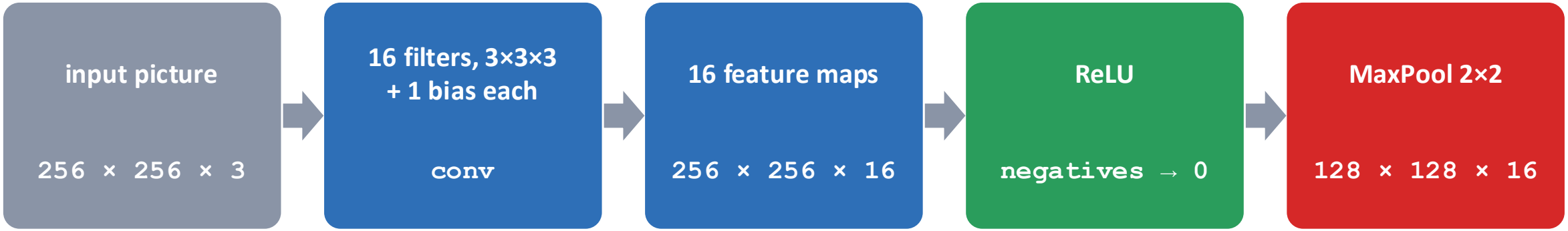
- negative numbers become 0, positive ones stay
- "no stripes here" and "anti-stripes here" both just mean 0
- without it, stacking layers would be no better than one layer

Max pooling: keep the strongest



- keeps the biggest number in each 2 × 2 square: half the width and height
- "there are stripes somewhere around here" is all we need
- next layer sees a wider area of the picture, and there's less to compute

One layer of a conv network, worked through



How many weights does it learn?

each filter: 3 × 3 × 3 = 27 weights + 1 bias = 28

16 filters: 16 × 28 = **448**

compared with 12.6 million for a fully connected layer

Wally example: one output number

- filter #5 learned "white above red"
- on Wally's shirt: 27 multiplications added up = 3.0, plus bias −0.5 = 2.5
- ReLU: 2.5 stays 2.5 (on a plain wall: −0.5 → 0)
- MaxPool: 2.5 survives if it's the biggest in its 2 × 2 square

Stack 4 blocks: from pixels to a map of descriptions

	shape (channels × height × width)	what it might detect
input	`3 × 256 × 256`	colours
block 1	`16 × 128 × 128`	edges, "red here"
block 2	`32 × 64 × 64`	stripes, corners
block 3	`64 × 32 × 32`	striped shirt, hat shape
block 4	`64 × 16 × 16`	whole-Wally parts

Only MaxPool shrinks the grid: 256 → 128 → 64 → 32 → 16.

The final 16 × 16 grid

- the picture cut into 256 squares, each 16 × 16 pixels
- each square has 64 numbers describing what's in it
- each square "sees" a patch about 46 px wide: big enough for a whole Wally
- the grid shrinks while the filters grow: we trade WHERE detail for WHAT detail

Classification vs localisation

Classification: WHAT is in it?



Wally? **yes 97%**
no 3%

- output: one probability per class (softmax)

Localisation: WHERE is it?



`(0.53, 0.14, 0.66, 0.36)`

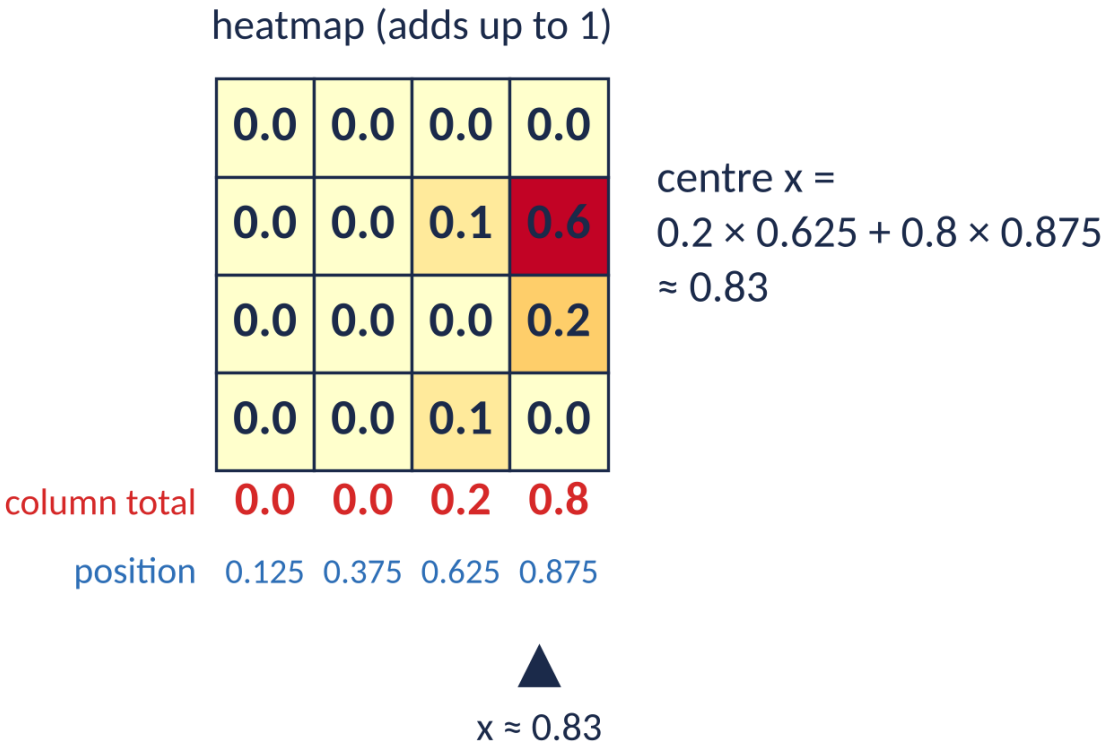
- output: 4 numbers, the box corners
- loss: mean squared error (how far off)
- today! Same CNN body, different head

PART 3

What's new in today's notebook

The heatmap head, the training loop and IoU

The heatmap head: "which square is Wally in?"



centre x =
$0.2 \times 0.625 + 0.8 \times 0.875$
≈ 0.83

1. **Score every square**

   a 1×1 conv asks each of the 256 squares "how Wally-like are you?"

2. **Softmax**

   turns the scores into probabilities that add up to 1: the heatmap

3. **Weighted average**

   of the square positions gives the centre (x, y)

4. **Size**

   the strongest response of each of the 64 channels → width and height

Why? The simple "flatten → Linear → 4 numbers" head just guessed the middle of every picture (IoU 0.03). One shared "is this Wally?" detector works anywhere.

Step 5: the training loop, 320 times over

1 Guess

```
predictions = model(images)
```

2 How wrong?

```
loss = loss_function(predictions, boxes)
```

3 Clear old gradients

```
optimizer.zero_grad()
```

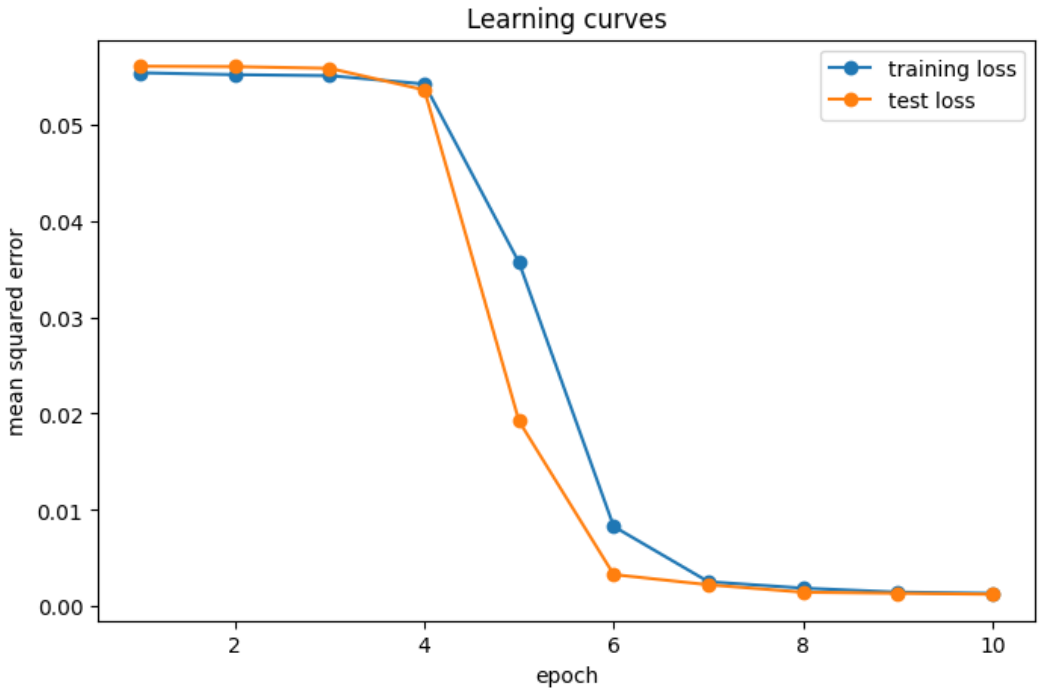
4 Work out the fix

```
loss.backward()
```

5 Nudge the weights

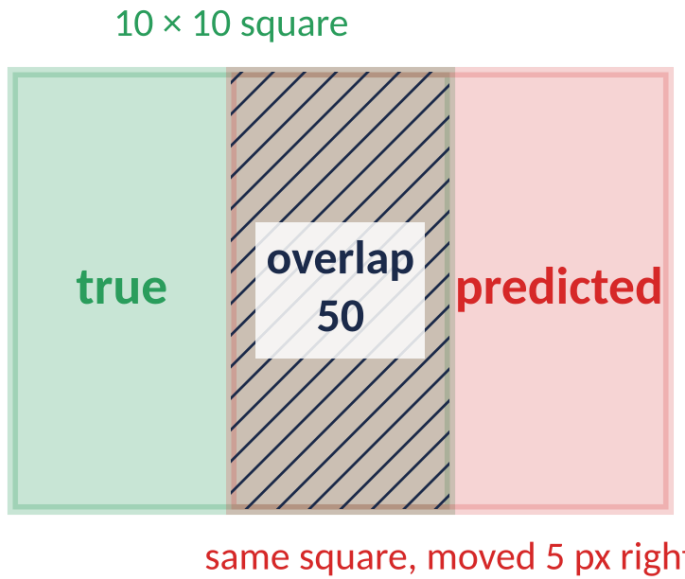
```
optimizer.step()
```

Learning rate (0.001) = how big each nudge is. Too big: it overshoots. Too small: it takes forever.



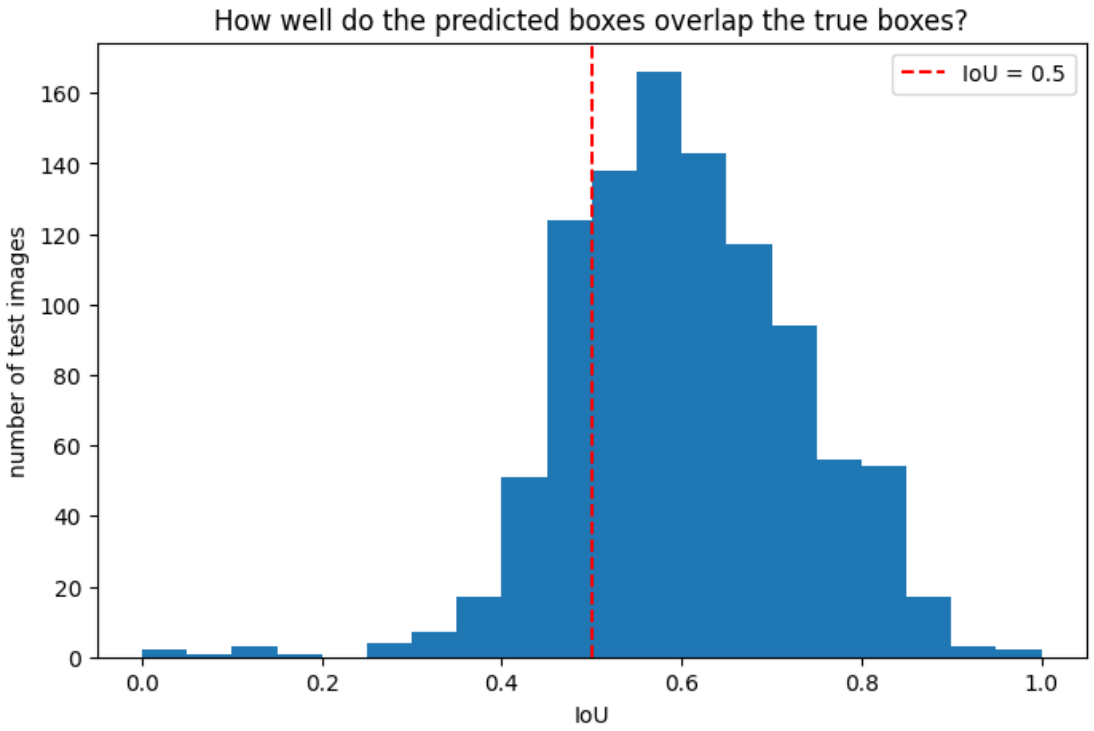
- **Both lines go down:** it's learning
- **Test much higher than training:** overfitting (memorising)
- **Flat at first?** normal here: it finds Wally around epoch 4–5

Step 6: scoring boxes with IoU



IoU = overlap ÷ total area covered

= 50 ÷ (100 + 100 − 50) = **0.33**



- **1** = perfect match, **0** = the boxes don't touch
- **IoU > 0.5** counts as "found Wally" (red dashed line)
- **IoU is strict:** just 5 px off on a small box already drops it to 0.33

What you should get by the end

0.60

mean IoU on 1000 unseen test pictures

79%

found (IoU > 0.5)

~2 min

training time on a Kaggle GPU



Where the model is looking: the bright squares of the heatmap on 4 test pictures it has never seen

Before you start

Turn on the GPU

Settings → Accelerator → GPU. Training takes about 2 minutes instead of 10+.

Run cells in order

Every cell uses variables from the ones above. Restarted? Run from the top.

Making 5000 pictures takes ~1 min

Run it once. Re-running wipes the folder and starts again.

Stuck? Print the shape

Most errors are shapes: (batch, channels, height, width).

Do the Think questions

Discuss them with your neighbour. They're the point of the workshop.

Finished early?

Try "What's next": more images, other learning rates, the flatten head.

Now go find Wally!

Questions? Then open the notebook and start at Step 0.



The model looking at a picture it has never seen.